

Giới thiệu về C++

Nguyễn Huyền Châu – AI Lab - TLU

Nội dung

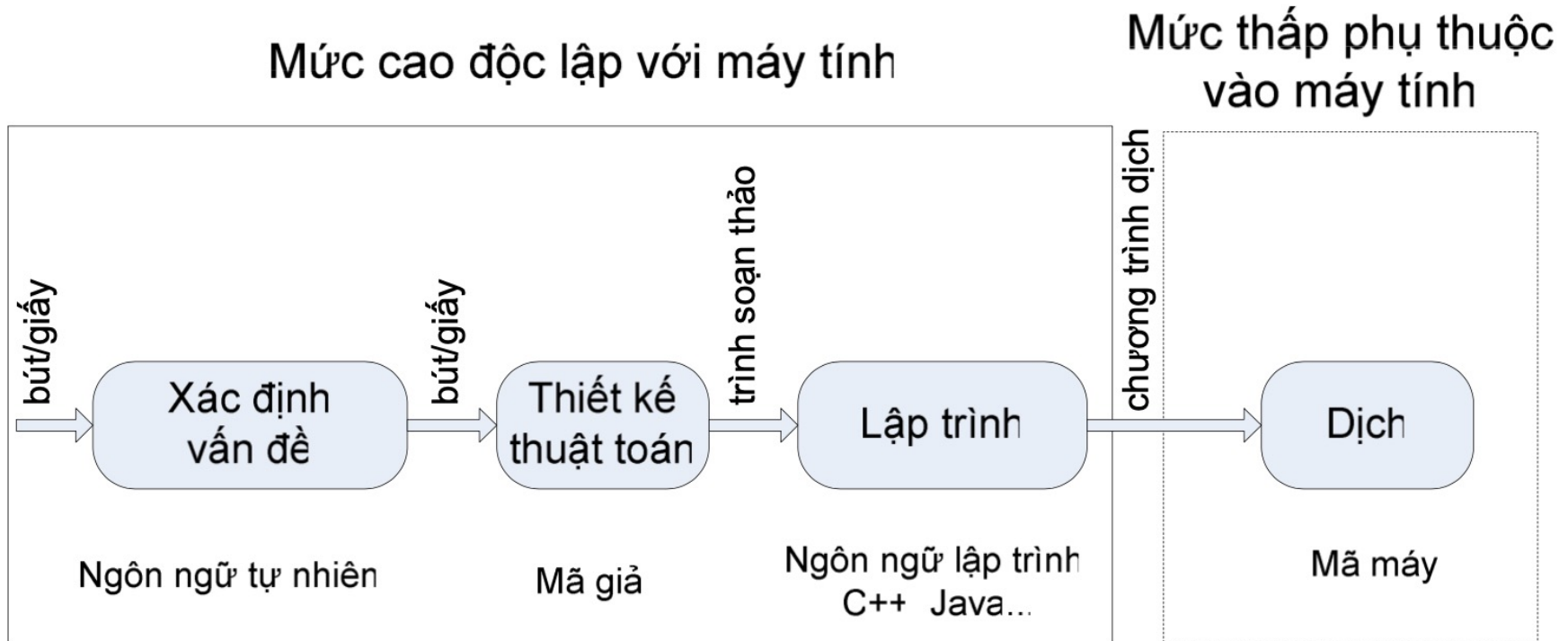
- Các khía cạnh của lập trình
- Cấu trúc chương trình C++
- Nghịch và gỡ rối
- Làm quen với cout
- Biến, nhập xuất biến
- Toán tử và biểu thức
- Các chủ đề mở rộng
- Trò chơi đoán số

Các khía cạnh của việc lập trình

- ❑ Lập trình, ngôn ngữ, chương trình dịch
- ❑ Các giai đoạn xây dựng chương trình
- ❑ Lập trình với C++ và chương trình đầu tiên

Lập trình là gì?

- Lập trình là viết chương trình, gồm 1 dãy chỉ thị ra lệnh cho máy
- Máy tính chỉ hiểu được các chỉ thị dạng mã máy
- Lập trình ngày nay không còn viết dạng mã máy
- Nó trở thành mức cao trong việc giải quyết bài toán.



Ngôn ngữ lập trình, chương trình dịch

- Ngôn ngữ bậc thấp: hợp ngữ, mã máy
 - máy tính hiểu được
 - chạy nhanh, song công kênh, khó viết, phụ thuộc phần cứng.
- Ngôn ngữ bậc cao: Python, C ++, Java,...
 - máy tính không hiểu, cần được dịch
 - chạy chậm hơn, nhưng đơn giản, dễ viết độc lập phần cứng,
 - biên dịch (compile): dịch tất một lúc
 - thông dịch (interpret): dịch từng lệnh
- C ++ là ngôn ngữ bậc cao và được biên dịch

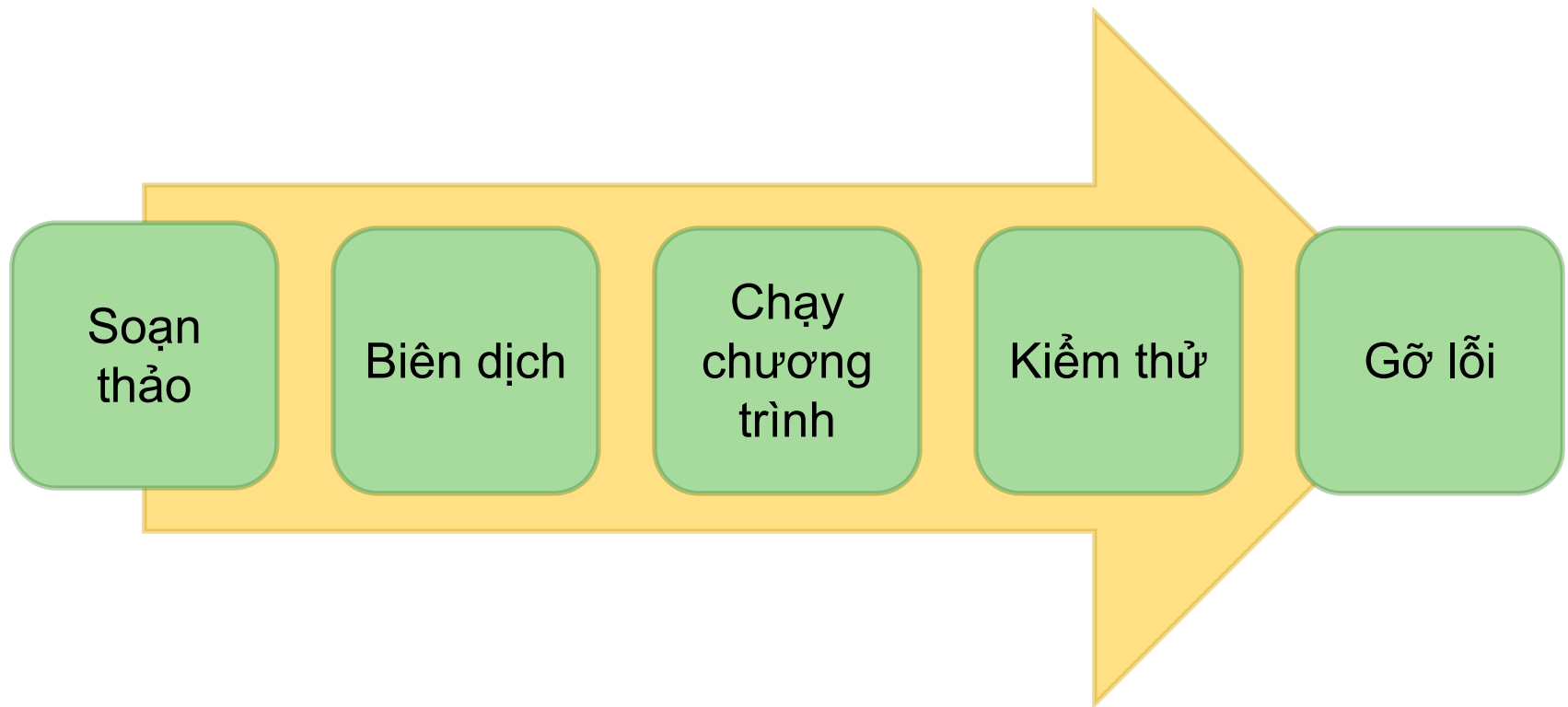
```
1010101010
0010111100
1010101010
```

LOAD	BASEPAY
ADD	OVERPAY
STORE	GROSSPAY

```
float total,price,tax;
total=price+tax
```

Các giai đoạn xây dựng chương trình

- Đi kèm các công cụ hỗ trợ:



Lập trình với C++

- Hệ thống lập trình C++ gồm:
 - Ngôn ngữ (gồm các mô tả về từ khoá, quy tắc cú pháp,...)
 - Môi trường lập trình (trình soạn thảo - editor, trình dịch - compiler, trình gỡ lỗi - debugger,...)
 - Thư viện chuẩn (các hàm, dữ liệu người khác viết sẵn)
- Môi trường lập trình C++:
 - Có thể dùng các công cụ riêng lẻ kết hợp với nhau
 - Hoặc dùng môi trường phát triển tích hợp (IDE) có sẵn mọi công cụ, vd: CodeBlocks, Visual Code, Dev C++,...
 - Hoặc chạy trên web, vd: cpp.sh, repl.it

Lập trình với C++ (tiếp)

- Buổi đầu: Làm trên repl.it (replit.com) hoặc cpp.sh
- Cuối giờ: Hướng dẫn cài Code::Blocks
- Về nhà: Hoàn thành nốt cài đặt + làm bài tập
- Gỡ rối cài đặt: tự tìm mạng + hỏi trợ giảng

Sinh viên có thể cài các editor và trình biên dịch C++ khác nếu thuận tiện

Ví dụ 1: Chương trình Hello World

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout<<"Hello World!\n";
7
8      return 0;
9  }
10 }
```

Tạo, biên dịch và chạy

- Tạo file, lưu **dưới tên .cpp**
- Biên dịch và chạy:
 - Web (cpp.sh hay repl.it): ấn Run
 - CodeBlocks: biên dịch Ctrl F9, chạy Ctrl F9, cả biên dịch và chạy dùng F9
 - Dòng lệnh: chuyển thư mục về thư mục chứa file .cpp
 - Gõ `make + <tên file bỏ .cpp>` để biên dịch
 - Gõ `./<tên file bỏ .cpp>` để chạy sau khi biên dịch

file .cpp và file .exe

- Biên dịch file .cpp (file nguồn) sẽ sinh ra thêm file .exe (file đích) chứa mã máy, do đó thực thi được (executable).



HelloWorld
Unix executable - 23 KB

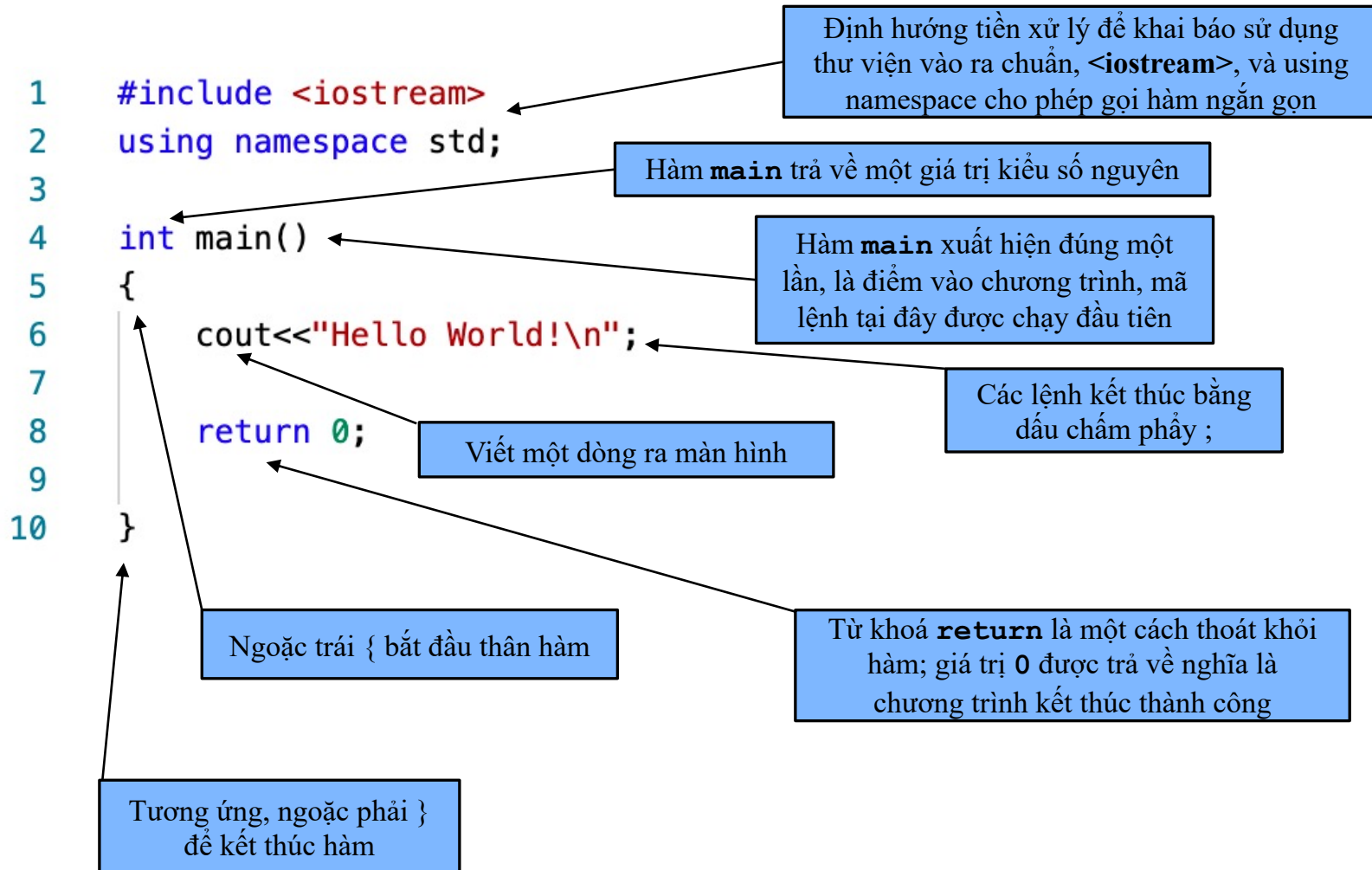
- File .exe chính là một ứng dụng, nên có thể click để chạy
 - chạy trên Window thì màn hình sẽ tự động bị đóng
 - để click mà ko đóng, thêm system ("pause") (học sau)



Cấu trúc một chương trình C++

- ❑ Các thành phần chương trình
- ❑ Nội dung hàm main()
- ❑ Các chỉ thị tiền xử lý

Ví dụ 1: Hello World



Chỉ thị tiền xử lý

- Các chỉ thị tiền xử lý được bắt đầu bằng #
 - Lưu ý kết thúc ***không có dấu ;***
 - Xử lý trước khi biên dịch nên trình biên dịch sẽ bỏ qua
- **#include <...>** là chỉ thị để khai báo thư viện <...>
 - Thư viện là một tập hợp các cấu trúc lập trình viết sẵn: dữ liệu, tài liệu, module, hàm, mã lệnh
 - Ai viết? Viết để làm gì?
- **<iostream>** là thư viện chuẩn, quản lý nhập xuất
 - In ra màn hình
 - Đọc từ bàn phím
 - ...

Nghịch và gỡ rối

- ❑ Chú thích
- ❑ Các dấu trắng
- ❑ Các lỗi lập trình
- ❑ Một lệnh hữu ích

Ví dụ 1 – nghịch 1

Kết quả?

the first program in C++

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout<<"Hello World!\n";
    return 0;
}
```

//the first program in C++

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout<<"Hello World!\n";
    return 0;
}
```

fatal error: too many errors emitted, stopping now [-ferror-limit=]
20 errors generated.

Ví dụ 1 – nghịch 1

```
/* -----  
Description: The first program in C++  
Input: None  
Output: Printing out a "Hello World" message  
----- */  
  
#include <iostream>  
using namespace std;  
  
int main()  
{  
    cout<<"Hello World!\n";  
    return 0;  
}
```

Chú thích (comment)

- Mô tả chương trình: mục đích của biến, đoạn mã, ... giúp dễ hiểu cho người khác lần bản thân về sau
- Sẽ được trình biên dịch bỏ qua
- Có thể dùng trong gỡ lỗi
- Chú thích trên 1 dòng:
`// some comment here`
- Chú thích trên nhiều dòng:
`/* some very long
comment here */`

Ví dụ 1 – nghịch 2 - các dấu trắng

Kết quả?

```
#include <iostream>
using namespace std;

int main()
{
    cout
    <<
    "Hello World!\n"
    ;

    return 0;
}
```

Hello World!

Thêm dấu trắng như space, tab, enter không thay đổi câu lệnh. Giúp mã dễ đọc, vd lệnh cout quá dài nên cắt thành nhiều dòng

Các lỗi lập trình

- Khi lập trình có thể sinh ra nhiều loại lỗi:
 - Lỗi biên dịch: chương trình không biên dịch được
 - Sai từ vựng, cú pháp, ngữ nghĩa của ngôn ngữ
 - Lỗi thời gian chạy: chương trình biên dịch được, nhưng có lỗi khi chạy
 - Tràn bộ nhớ, truy cập phần bộ nhớ không được phép, lỗi chia cho 0
 - Lỗi logic: biên dịch được, chạy được, kết quả sai
 - Tính trung bình cộng 3 số nhưng lấy tổng chia 2
- Ta tập trung mô tả lỗi biên dịch

Ví dụ 1 – nghịch 3

Kết quả?

```
1  // #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello World!\n";
7
8      return 0;
9
10 }
```

1CommentInclude.cpp:6:5: **error:** use of undeclared identifier 'cout'

cout << "Hello World!\n";

^

1 warning and 1 error generated.

Ví dụ 1 – nghịch 4

Kết quả?

```
#include <iostream>
using namespace std;

int main()
{
    c<<"Hello World!\n";
    return 0;
}
```

```
HWcoutToC.cpp:5:5: error: use of undeclared identifier 'c'
    c << "Hello World!\n";
    ^
```

1 error generated.

make: *** [HWcoutToC] Error 1

Ví dụ 1 – nghịch 5

Kết quả?

```
1  #include <iostream>
2  //using namespace std;
3
4  int main()
5  {
6      cout << "Hello World!\n";
7
8      return 0;
9
10 }
```

```
1CommentInclude.cpp:6:5: error: use of undeclared identifier 'cout'; did you mean 'std::cout'?
    cout << "Hello World!\n";
    ~~~~
    std::cout
```

Ví dụ 1 – nghịch 6

Kết quả?

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      cout<<"Hello World!\n"
6      return 0;
7  }
```

```
HWmissSemiComa.cpp:5:29: error: expected ';' after expression
    cout << "Hello World!\n"
                           ^
```

```
;
```

```
1 error generated.
```

```
make: *** [HWmissSemiComa] Error 1
```

Bài tập: Nghịch

- Hãy sửa đổi các dòng lệnh chương trình HelloWorld để sinh ra các loại lỗi khác nhau và thử giải thích ý nghĩa của các thông báo lỗi tương ứng.

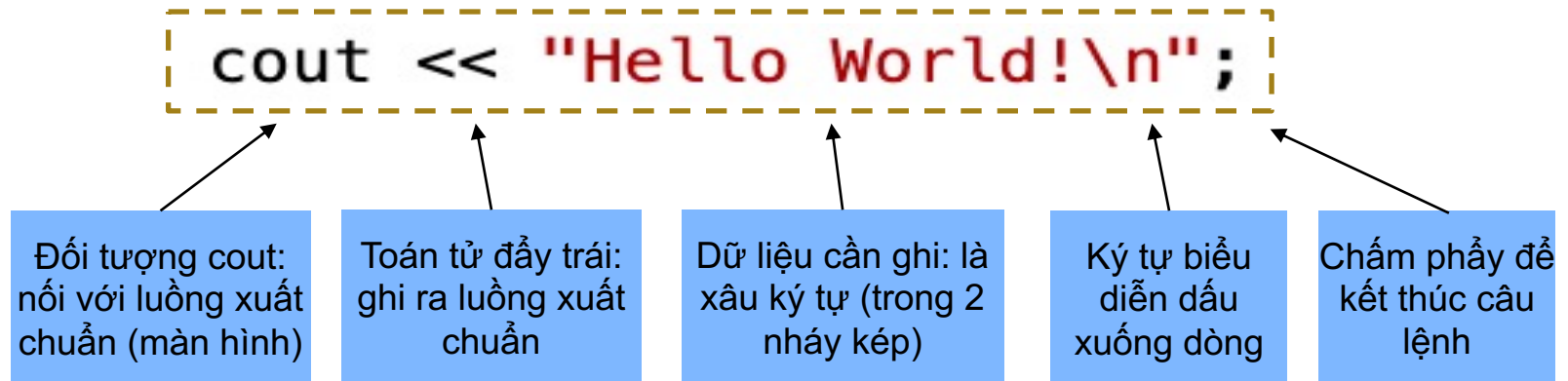
Một hàm hữu ích: `system("pause")`

- Khi click file .exe trên máy Windows, cửa sổ chương trình sẽ đóng ngay sau khi chạy xong, ta không kịp nhìn kết quả. Để dừng màn hình có 2 cách:
 - Mở sẵn console trước khi click exe
 - Trong chương trình có dùng lệnh `system("pause")`
- Nó thuộc thư viện `<cstdlib>` // cần `#include <cstdlib>`
- Muốn hết dừng thì ấn Enter, sẽ tiếp tục với lệnh sau đó, nếu không có lệnh nào thì sẽ đóng màn hình
- Chỉ dùng được trên Windows

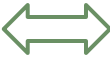
Làm quen với cout

- ❑ Cú pháp cout
- ❑ Ký tự thoát

Cú pháp cout



- Câu lệnh trên thực chất là một biểu thức:
 - ❑ dấu `<<` là toán tử - tựa như phép toán `+`, `-`, `*`, `-`
 - ❑ `cout`, `"Hello World!\n"` là các toán hạng, giống các số hạng của phép toán
 - ❑ biểu thức này có kết quả chính là `cout`, nên lại kết hợp được `<<` để in tiếp các chuỗi, gọi là dùng `cout` kiểu lợp ngói – cascading `cout`

`cout << "something" << "another thing";`  `cout << "something";`
`cout << "another thing";`
 - ❑ Giống như `2 + 5` là biểu thức cộng 2 số nguyên, có kết quả cũng là 1 số nguyên, nên lại `+` được tiếp: `2+5+8 ...`

Ký tự thoát (escape characters)

Chuỗi Escape	Mô tả
<code>\n</code>	Dòng mới. Đặt con trỏ màn hình tại đầu dòng tiếp theo.
<code>\t</code>	Tab. Di chuyển con trỏ đến điểm dừng tab tiếp theo.
<code>\r</code>	Về đầu dòng. Chuyển con trỏ màn hình tới đầu dòng hiện tại; không xuống dòng mới.
<code>\a</code>	Chuông. Bật chuông hệ thống.
<code>\\</code>	Chéo ngược. Dùng để in một dấu chéo ngược.
<code>\"</code>	Nháy kép. Dùng để in một dấu nháy kép.

Ví dụ 2a: Nhiều lệnh mà in ra một dòng

Kết quả?

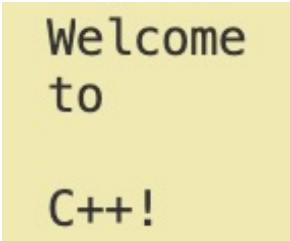
```
1  // Printing a line with multiple statements
2  #include <iostream>
3
4  using namespace std;
5
6  int main()
7  {
8      cout << "Welcome ";
9      cout << "to C++!\n";
10
11     return 0;
12 }
```

Welcome to C++!

Ví dụ 2b: Một lệnh mà in ra nhiều dòng

Kết quả?

```
1  // Printing multiple lines with a single statement
2  #include <iostream>
3
4  using namespace std;
5
6  int main()
7  {
8      cout << "Welcome\nto\n\nC++!\n";
9
10     return 0;
11 }
```



Welcome
to
C++!

Ví dụ 2c: Các kiểu ký tự thoát

Kết quả?

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      cout << "Print out a tab \t then a slash \\"
8          << " then a return \r"
9          << " then an endl\n"
10         << "and finally an apostrophe \'";
11
12     return 0;
13 }
```

```
then an endl          then a slash \ then a return
and finally an apostrophe 'Chaus-MacBook-Pro:cpp chaunguyen$
```


Thử viết chương trình đầu tiên

- ❑ Tự in ra họ, tên của bạn trên các dòng khác nhau
- ❑ In ra họ tên, nhưng trên một dòng, cách nhau bởi dấu tab

Một bài tập nữa

- Viết một chương trình "vẽ" ra tên bạn bằng các ký tự * trên nhiều dòng, ở dạng sau – vd với tên Mai

```

*      *      *      *
**     **     *  *    *
*  *  *  *    *      *  *
*    *    *   **** *  *
*          *  *           *  *
```

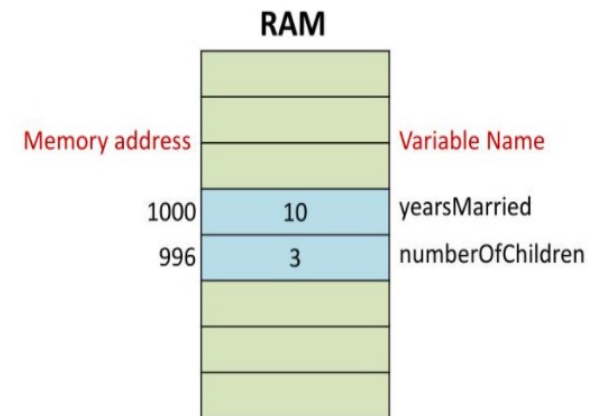
Biến và nhập xuất

- ❑ Khai báo biến
- ❑ Kiểu dữ liệu
- ❑ Cách đặt tên
- ❑ Nhập xuất biến với cin/cout
- ❑ Hoạt động cơ bản của biến

Khái niệm biến

- Trong máy tính, dữ liệu được lưu trong các vùng nhớ có đánh địa chỉ. Song số lượng địa chỉ rất lớn và khó nhớ
 - bộ nhớ 4GB RAM đã xấp xỉ 32 tỷ bit
- C++ cho phép đặt tên và gán kiểu dữ liệu cho vùng nhớ
 - Truy cập bằng tên tiện lợi hơn địa chỉ và tên còn giúp gợi nhắc ý nghĩa
 - Gán kiểu giúp quản lý biến tốt hơn

Biến đơn giản là một vùng nhớ được đặt tên và gán kiểu



Ví dụ 3a: Khai báo biến

```
1 // Enter and print out the radius of a circle
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     float radius;
8     cout << "Please enter the radius ";
9     cin >> radius;
10    cout << "The radius you've enter is: " << radius << endl;
11
12    return 0;
13 }
```

Kiểu của biến

Tên biến

Khai báo biến

- Biến cần được khai báo trước khi dùng theo cú pháp:
 <kiểu dữ liệu> <tên biến>;
 int num; float radius;
- Biến tồn tại trong phạm vi từ dòng khai báo cho đến hết khối lệnh chứa khai báo.

```
int n=7;  
{  
    int m = 5;  
    n=m;  
}  
cout << "m = " << m  
    << "and n = " << n << endl;
```



Oops, out of scope!!!

Các kiểu cơ bản

- Các kiểu cơ bản: Kiểu số nguyên (`char`, `short`, `int`, `long`), kiểu ký tự (`char`), kiểu số thực hay kiểu chấm động (`float`, `double`, `long double`), kiểu logic (`bool`)
- Các kiểu dẫn xuất được xây từ các kiểu cơ bản nhờ các toán tử đặc biệt hoặc cơ chế struct và class (học sau).
 - Một kiểu rất hữu ích là kiểu xâu
- Các biến, hằng cùng kiểu được khai báo trong một lệnh.
`float` length, width, area;
`const float` PI =3.14, E=2.718;

Các kiểu cơ bản (tiếp)

- Các kiểu chủ yếu khác nhau ở kích thước lưu trữ, kéo theo khác nhau về phạm vi biểu diễn và độ chính xác
- Kích thước một kiểu thay đổi tùy hệ thống máy tính, nhưng:
 - char luôn có cỡ 1 byte
 - char, short, int, long phải có cỡ tăng dần
 - float, double, long double có cỡ lẫn độ chính xác tăng dần
 - các kiểu nguyên không dấu chỉ chứa số dương, song biểu diễn nhiều số dương gấp đôi kiểu nguyên có dấu tương ứng
 - các kiểu thực thì luôn có dấu
 - bool được lưu như số nguyên nhỏ, với giá trị True/False được lưu như 1/0.
- Hàm sizeof(<tên kiểu hoặc tên biến>) để tính cỡ kiểu, cỡ biến

Các kiểu cơ bản (tiếp)

Kiểu	Mô tả	Kích thước thông dụng (byte)	Phạm vi (tương ứng với kích thước)
char	ký tự / số nguyên nhỏ	1	các kí tự ASCII signed char: -128 → 127, hoặc unsigned char: 0 → 255
bool	giá trị Boolean	1	true hoặc false
short	số nguyên	2	signed short: -32767 → 32767 unsigned short: 0 → 65536
int	số nguyên lớn	4	signed int: 2147483648 → 2147483647 unsigned int: 0 → 4294967296
long	số nguyên rất lớn	4	signed long: 2147483648 → 2147483647 unsigned long: 0 → 4294967296
float	số thực	4	+/- 1.4023x10 ⁻⁴⁵ → 3.4028x10 ⁺³⁸
double	số thực với độ chính xác cao	8	+/- 4.9406x10 ⁻³²⁴ → 1.7977x10 ³⁰⁸
long double	số thực với độ chính xác rất cao	8	+/- 4.9406x10 ⁻³²⁴ → 1.7977x10 ³⁰⁸

Ví dụ 3b: Các kiểu cơ bản

```
char c;
cout << "A char is stored in " << sizeof (char) << " bytes \n";
cout << "c variable is stored in " << sizeof (c) << " bytes \n";
bool b;
cout << "A bool is stored in " << sizeof (bool) << " bytes \n";
cout << "b variable is stored in " << sizeof (b) << " bytes \n";
short shortInt;
cout << "A short is stored in " << sizeof (short) << " bytes \n";
cout << "shortInt variable is stored in " << sizeof (shortInt) << " bytes \n";
unsigned short unsignedShort;
cout << "An unsigned short is stored in " << sizeof (unsigned short) << " bytes \n";
cout << "unsignedShort variable is stored in " << sizeof (unsignedShort) << " bytes \n";
int i;
cout << "A int is stored in " << sizeof (int) << " bytes \n";
cout << "i variable is stored in " << sizeof (i) << " bytes \n";
unsigned int unsignedInt;
cout << "A unsigned int is stored in " << sizeof (unsigned int) << " bytes \n";
cout << "A unsignedInt is stored in " << sizeof (unsignedInt) << " bytes \n";
long l;
cout << "A long is stored in " << sizeof (long) << " bytes \n";
cout << "l variable is stored in " << sizeof (l) << " bytes \n";
unsigned long unsignedLong;
cout << "An unsigned long is stored in " << sizeof (unsigned long) << " bytes \n";
cout << "unsignedLong variable is stored in " << sizeof (unsignedLong) << " bytes \n";
float f;
cout << "A float is stored in " << sizeof (float) << " bytes \n";
cout << "f variable is stored in " << sizeof (f) << " bytes \n";
double d;
cout << "A double is stored in " << sizeof (double) << " bytes \n";
cout << "d variable is stored in " << sizeof (d) << " bytes \n";
long double longDouble;
cout << "A long double is stored in " << sizeof (long double) << " bytes \n";
cout << "longDouble variable is stored in " << sizeof (longDouble) << " bytes \n";
return 0;
```

A char is stored in 1 bytes
c variable is stored in 1 bytes
A bool is stored in 1 bytes
b variable is stored in 1 bytes
A short is stored in 2 bytes
shortInt variable is stored in 2 bytes
An unsigned short is stored in 2 bytes
unsignedShort variable is stored in 2 bytes
A int is stored in 4 bytes
i variable is stored in 4 bytes
A unsigned int is stored in 4 bytes
A unsignedInt is stored in 4 bytes
A long is stored in 8 bytes
l variable is stored in 8 bytes
An unsigned long is stored in 8 bytes
unsignedLong variable is stored in 8 bytes
A float is stored in 4 bytes
f variable is stored in 4 bytes
A double is stored in 8 bytes
d variable is stored in 8 bytes
A long double is stored in 16 bytes
longDouble variable is stored in 16 bytes

Quy tắc đặt tên (định danh)

- Không chỉ biến, nhiều thành phần khác trong chương trình cũng có thể được đặt tên như hàm, lớp,...
- Mọi tên/định danh phải là duy nhất
- Chỉ gồm chữ cái a..z, A..Z, chữ số 0..9, dấu gạch dưới _
- Nhưng không bắt đầu bằng chữ số
- Không trùng tên các định danh khác
- Không được trùng các từ khóa của C++
 - `int`, `if`, `return`, ...
- Phân biệt chữ hoa chữ thường:
 - `number` khác với `Number`

Các từ khoá cần tránh

alignas alignof and and_eq asm atomic_cancel atomic_commit atomic_noexcept auto bitand	bitor bool break case catch char char16_t char32_t class compl	concept const constexpr const_cast continue decltype default delete do double	dynamic_cast else enum explicit export extern false float for friend
---	---	--	---

goto if import inline int long module mutable namespace new noexcept	not not_eq nullptr operator or or_eq private protected public register reinterpret_cast	requires return short signed sizeof static static_assert static_cast struct switch synchronized	template this thread_local throw true try typedef typeid typename union unsigned	using virtual void volatile wchar_t while xor xor_eq
--	---	---	--	---

Cách đặt tên tốt

- Chọn tên có ý nghĩa, gợi nhớ công dụng, kiểu dữ liệu

```
area = width * height; //meaningful names  
sum = integer1 + integer2; //type signaling names  
x = y * z; //not good names
```

- Tên viết tắt thường khó đọc cho người khác, lẫn người lập trình sau 1 ngày, 1 tuần
- Chỉ viết tắt khi tên thông dụng, ai cũng hiểu
 - toạ độ x, y, bán kính r
 - Biến đếm i

Hãy lập trình giống như kể chuyện

Cách đặt tên tốt

- Không viết tắt song cũng không quá dài:
 - Tên tồi: `theNumberOfPeopleWhoContractedCovid`
 - Tên tốt: `covidCasesNumber`
- Với tên gồm nhiều từ thì có thể chọn:
 - Kiểu gạch dưới: `first_name`
 - Kiểu “lạc đà” (CamelCase): `firstName`
- Làm sao để dễ phân biệt biến và hằng?
 - Kiểu lạc đà cho biến, kiểu viết hoa tất và gạch dưới cho hằng: `circleRadius`, `GOLD_RATIO`
- Duy trì phong cách thống nhất trong chương trình

Ví dụ 3c: Nhập biến với cin

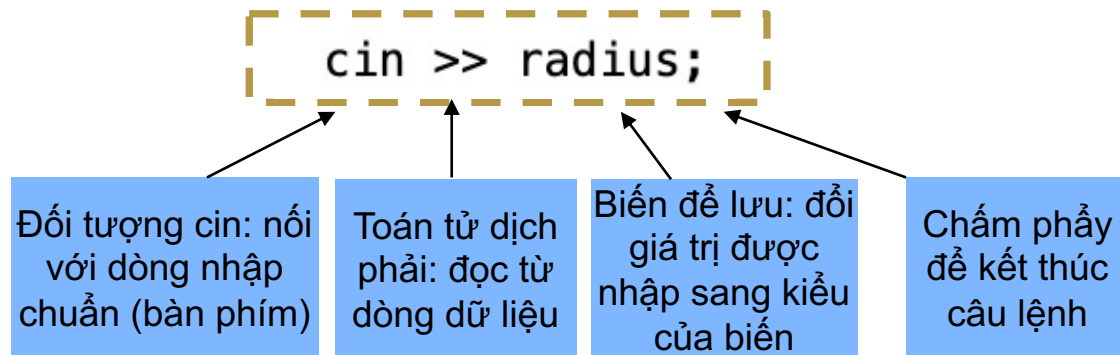
```
// Enter a radius of a circle and print out the radius and the area
#include<iostream>
using namespace std;
```

```
int main(){
    float radius;
    cout << "Radius: ";
    cin >> radius;
    float area = 3.14 * radius * radius;
    cout << "Radius: " << radius << endl << "Area: " << area << endl;
}
```

Biến

Lệnh đọc giá trị từ bàn phím vào biến

Nhập biến với cin



- Toán tử `>>` kết hợp với `cin` có thể đọc dòng dữ liệu nhập từ bàn phím
 - Nó đợi người dùng nhập giá trị khác trắng, rồi đợi phím Enter.
 - Rồi đọc dòng dữ liệu đến khi gặp khoảng trắng (dấu cách, tab, xuống dòng)
 - Lưu giá trị đọc được vào biến ở bên phải dấu `>>`, đổi giá trị thành kiểu biến
- Tương tự lệnh xuất, lệnh nhập cũng tạo thành một biểu thức, tại đó:
 - `>>` là toán tử; và `cin`, `radius` là các toán hạng
 - kết quả cũng là `cin`, nên lại kết hợp được với `>>` để đọc tiếp dữ liệu

`cin >> integer1 >> integer2;`  `cin >> integer1;`
`cin >> integer2;`

Ví dụ 3d: Xuất biến với cout

```
// Enter a radius of a circle and print out the radius and the area
#include<iostream>
using namespace std;

int main(){
    float radius;
    cout << "Radius: ";
    cin >> radius;
    float area = 3.14 * radius * radius;
    cout << "Radius: " << radius << endl << "Area: " << area << endl;
}
```

cin nhập giá trị cho radius

cout xuất giá trị radius ra màn hình, đồng thời trả về kết quả lại là cout để xuất tiếp endl, ...

Cú pháp cout (ôn lại)

```
cout << "Radius: " << radius << endl << "Area: " << area << endl;
```

- Câu lệnh trên thực chất là một biểu thức:
 - dấu << là toán tử - tựa như phép toán +, -, *, -
 - cout, "Radius: " là 2 toán hạng
 - cout << "Radius: " là một biểu thức có giá trị bằng chính cout, nên lại kết hợp được << để in tiếp radius, rồi cứ thế in tiếp endl, "Area: ", area, và endl theo kiểu lợp ngói.

Hoạt động của biến

- 2 hoạt động chính là đọc /ghi vùng nhớ của biến
 - đọc là lấy ra giá trị từ biến (để in, để tính toán), ghi là điền giá trị vào biến (nhập vào, gán cho, biến đổi)
 - đọc biến không làm thay đổi vùng nhớ
 - ghi biến thì giá trị cũ bị ghi đè

```
a = 5; //a = 5
```

```
b = 2; //b = 2
```

```
b = a; //b = 5, a = 5
```

```
b = b + 2 //b = 7, a = 5
```

Hoạt động của biến (tiếp)

`cin >> integer1`

- giả sử người dùng nhập 45

<code>integer1</code>	45
-----------------------	----

`cin >> integer2`

- giả sử người dùng nhập 72

<code>integer1</code>	45
<code>integer2</code>	72

`sum = integer1 + integer2`

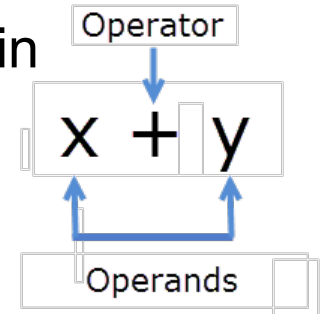
<code>integer1</code>	45
<code>integer2</code>	72
<code>sum</code>	117

Toán tử và biểu thức

- ❑ Toán tử gán
- ❑ Toán tử số học
- ❑ Toán tử gán kết hợp
- ❑ Thứ tự kết hợp các toán tử

Toán tử - Biểu thức

- Toán tử là một thao tác trên các thành phần thông tin
 - các thành phần gọi là các toán hạng của toán tử
 - Toán tử gán, nhân số, cộng xâu, và/hoặc logic, đọc/ghi dữ liệu, ...
- Toán tử có yêu cầu về kiểu với toán hạng. Một kiểu chỉ kết hợp được với vài toán tử nhất định



```
int1 + float2 + double3 //valid    int1 + string2 //invalid
string1 + string2 //valid          string1 && string2 //invalid
```

- Toán tử kết hợp toán hạng tạo thành biểu thức, bản thân biểu thức cũng mang kiểu và có giá trị

```
firstName = "Huyen Chau", surName = "Nguyen";
fullName = firstName + surName; //an expression of string type
```

Toán tử gán và biểu thức gán

- Quy trình gán: Tính xong vế phải, được bao nhiêu ghi vào vế trái
 - nên vế trái phải là biến
- Dùng cho mọi kiểu cơ bản lẫn kiểu string, miễn hai vế cùng kiểu
- Gán lúc khai báo gọi là khởi gán
- Biểu thức gán có kiểu và giá trị trùng với toán hạng trái, nên gán tiếp được (~ cin/cout cascading)

```
a = a + 3; // valid
3 = a + 3; // error
a + 3 = 5 // error
```

```
varInt=5;
varFloat=3.5;
varChar='E' ;
varBool=false;
varString = "Hello, World";
int varInt=5;
```

```
cout << (a = b = c = d = 5);
//all numbers are 5
//also print out 5
```

Ví dụ 4a: Giới thiệu toán tử số học

```
1  #include <iostream>
2  using namespace std;
3
4  ✓ int main()
5  {
6      int a, b;
7      cout << "Enter a: ";
8      cin >> a;
9      cout << "Enter b: ";
10     cin >> b;
11
12     cout << a << " + " << b << " = " << a + b << endl;
13     cout << a << " - " << b << " = " << a - b << endl;
14     cout << a << " * " << b << " = " << a * b << endl;
15     cout << a << " / " << b << " = " << a / b << endl;
16     cout << a << " % " << b << " = " << a % b << endl;
17     return 0;
18 }
```

```
Enter a: 3
Enter b: 5
3 + 5 = 8
3 - 5 = -2
3 * 5 = 15
3 / 5 = 0
3 % 5 = 3
```

Phong cách lập trình tốt: Đặt dấu cách trước và sau các toán tử

Các toán tử số học cơ bản

- Toán tử 1 ngôi chỉ có phép - (âm, trừ), vd -3, -x,...
- Toán tử 2 ngôi là các phép tính:
 - + (cộng), - (trừ), * (nhân), / (chia), % (lấy phần dư)
 - +, -, *, / dùng cho các toán hạng số cùng kiểu
 - % chỉ dùng cho toán hạng số nguyên cùng kiểu
 - *Các ràng buộc xét theo kiểu, không theo giá trị*
- Biểu thức kết quả có kiểu là kiểu của toán hạng tham gia biểu thức, và có giá trị tính từ các quy tắc toán học (chú ý phân biệt giá trị biểu thức chia nguyên vs. chia thực)
- Để check kiểu của biểu thức bất kỳ dùng hàm `typeid(<biểu thức>).name()` trong thư viện `<typeinfo>`

Ví dụ 4b: Phép toán kiểu nguyên vs. thực

```
1  ✓ #include <iostream>
2  #include <typeinfo>
3  using namespace std;
4
5  ✓ int main()
6  {
7      int int1=5, int2=2;
8      float float1=6, float2=2;
9
10     cout << "For integers:\n";
11     cout << int1 << " / " << int2 << " = " << (int1/int2)
12         << " and of type " << typeid(int1/int2).name() << endl;
13     cout << int1 << " % " << int2 << " = " << (int1%int2)
14         << " and of type " << typeid(int1%int2).name() << endl;
15
16     cout << "For floats:\n";
17     cout << float1 << " / " << float2 << " = " << (float1/float2)
18         << " and of type " << typeid(float1/float2).name() << endl;
19  ✓ // cout << float1 << " % " << float2 << " = " << (float1%float2)
20     // << " and of type " << typeid(float1%float2).name() << endl;
21     // the statement above will create error
22
23     return 0;
24 }
```

Khai báo sử dụng thư viện <typeinfo>

Phép chia nguyên cho
biểu thức kiểu nguyên

For integers:
5 / 2 = 2 and of type i
5 % 2 = 1 and of type i
For floats:
6 / 2 = 3 and of type f

Phép chia thực cho biểu thức
kiểu thực, cả khi chia hết

Ràng buộc trên kiểu,
không trên giá trị: Kiểu
thực giá trị nguyên vẫn lỗi

Thứ tự kết hợp toán tử số học

- Quy tắc:
 - Tính trong ngoặc trước
 - Nhiều ngoặc thì ngoặc trong tính trước ngoặc ngoài
 - Phép (-) một ngôi mạnh hơn các phép hai ngôi
 - Phép *, /, % trước rồi +, - sau
 - Cùng mức thì tính từ trái qua phải
- Biểu thức sau tính thế nào?

```
expression = first * (second - first % 3) + first / 10 % 2 ;
```

Ví dụ 4c: Thứ tự kết hợp

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int first, second, expression;
6
7      cout<<"First integer: ";
8      cin>>first;
9      cout<<"Second integer: ";
10     cin>>second;
11
12     expression = (first + second % 3) / 2 - first * 5 % 3;
13
14     cout<< "(" << first << " + " << second << " % 3) / 2 - "
15         << first << " * 5 % 3 = " << expression << endl;
16
17     return 0;
18 }
```

First integer: 2
Second integer: 5
 $(2 + 5 \% 3) / 2 - 2 * 5 \% 3 = 1$

Các toán tử gán kết hợp

Toán tử	Ví dụ	Ý nghĩa
<code>+=</code>	<code>x += 5;</code>	<code>x = x + 5;</code>
<code>-=</code>	<code>y -= 2;</code>	<code>y = y - 2;</code>
<code>*=</code>	<code>z += 10;</code>	<code>z = z + 10;</code>
<code>/=</code>	<code>a /= b;</code>	<code>a = a / b;</code>
<code>%=</code>	<code>c %= 3;</code>	<code>c = c % 3;</code>

- Biểu thức có kiểu là kiểu của toán hạng trái, còn giá trị thì được tính tùy quy tắc của phép toán lẫn phép gán

Mở rộng: Hằng

- ❑ Hằng trực tiếp, hằng đặt tên
- ❑ Cách định kiểu cho biến và hằng

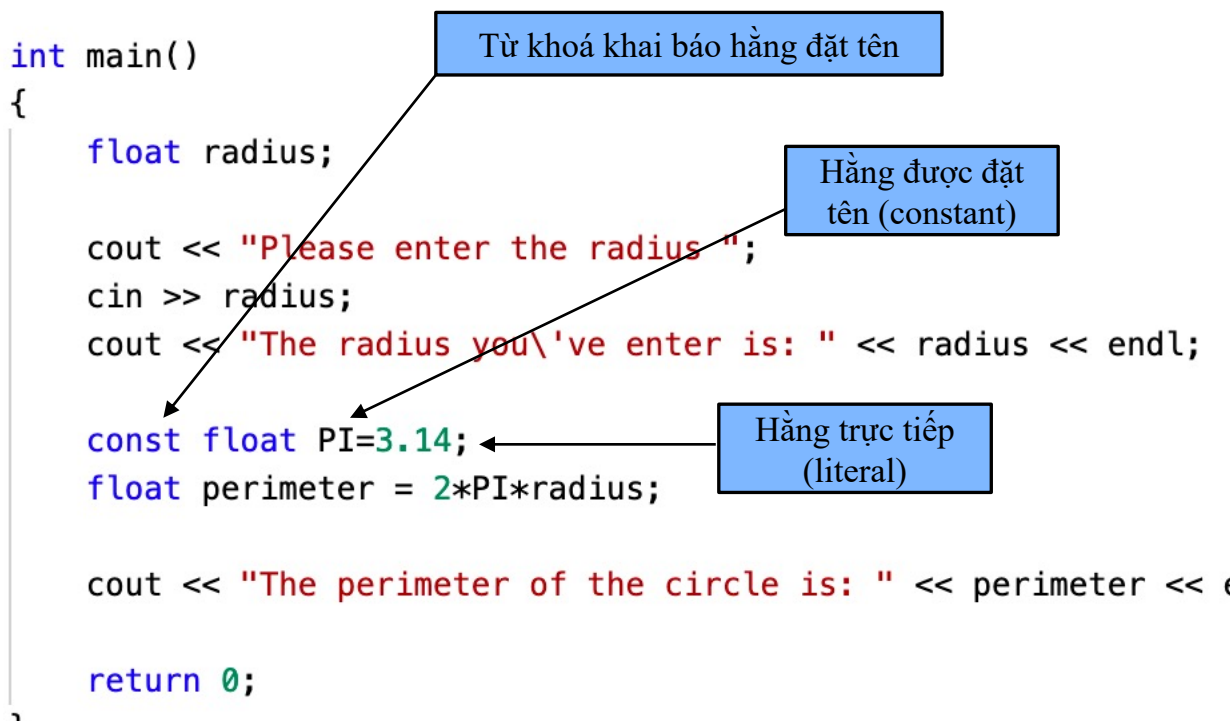
Ví dụ 5: Nhập bán kính, tính chu vi

```
1  // Enter a radius and print out the perimeter of the circle
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      float radius;
8
9      cout << "Please enter the radius ";
10     cin >> radius;
11     cout << "The radius you've enter is: " << radius << endl;
12
13     const float PI=3.14;
14     float perimeter = 2*PI*radius;
15
16     cout << "The perimeter of the circle is: " << perimeter << endl;
17
18     return 0;
19 }
20
```

Từ khoá khai báo hằng đặt tên

Hằng được đặt tên (constant)

Hằng trực tiếp (literal)



Hằng giá trị (literal), hằng đặt tên (const)

- Literal: các giá trị cụ thể không thể thay đổi, dùng trực tiếp trong chương trình. VD: 3, 3.14, 'A', "Hello", true
- Hằng đặt tên (hằng): Cũng biểu diễn giá trị cụ thể không thể thay đổi, nhưng gọi thông qua tên thay vì viết trực tiếp:
 - Cần khai báo tên, kiểu, và PHẢI ĐƯỢC KHỞI GÁN
 - Tương tự khai báo biến song thêm từ khoá **const** :
const <kiểu> <tên hằng>=<giá trị khởi tạo>;
const float PI=3.14;
 - Tại sao bắt buộc phải khởi tạo hằng?
 - Tại sao nên dùng hằng PI thay vì literal 3.14?
 - Làm sao nhớ đâu là hằng để không lỡ thay đổi nó?

Định kiểu cho biến, hằng, literal

- Biến và hằng luôn giữ kiểu như khai báo ban đầu
- Literal số nguyên mặc định hiểu là kiểu int và hệ 10, nếu muốn đổi cần thêm các chỉ định:
 - 1234L sẽ lưu 1234 như kiểu long; 075 sẽ lưu 75 dưới hệ 8; và 0x75A lưu 75A dưới hệ 16.
- Literal số thực (dạng chấm tĩnh như 3.14 hay dạng ký pháp khoa học như 1.45E10) thì luôn mặc định hiểu là double, dù có thể chỉ định thành float (3.14159f) hay thành long double (0.0039L)
- Literal ký tự luôn hiểu là kiểu char;
- Literal **true/false** luôn hiểu là kiểu bool

Mở rộng: Phong cách lập trình

- ❑ Vài quy tắc cơ bản
- ❑ Một chương trình ví dụ

Phong cách lập trình

- Tập hợp các cách tổ chức phần nhìn của mã nguồn
 - Không thay đổi chức năng chương trình
 - Giúp tăng tính dễ đọc
- Biết cách dùng khoảng cách, tab, khoảng trắng
 - Tab để căn lề
 - Trước sau mỗi toán tử có dấu cách
 - Gom các đoạn mã cùng mục đích, phân tách với các phần khác, vd tách phần khai báo và các lệnh khác bằng dòng trắng

Phong cách lập trình

- Giống ngoặc { } theo chiều dọc, vào trong ngoặc từ đó nhớ nhớ lùi vào một mức tab
- Lệnh dài nên chia thành nhiều dòng, giống theo toán tử
- Chú thích hợp lý, đặt tên gọi tả

Ví dụ 6: Phong cách lập trình

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  int main()
6  {
7      int num1 = 2897, num2 = 5, num3 = 837, num4 = 34, num5 = 7,
8          num6 = 1623, num7 = 390, num8 = 3456, num9 = 12;
9
10     // Display the first row of numbers
11     cout << setw(8) << num1 << " "
12         << setw(8) << num2 << " "
13         << setw(8) << num3 << endl;
14     // Display the second row of numbers
15     cout << setw(8) << num4 << " "
16         << setw(8) << num5 << " "
17         << setw(8) << num6 << endl;
18     // Display the third row of numbers
19     cout << setw(5) << setw(4) << setw(8) << num7 << " "
20         << setw(8) << num8 << " "
21         << setw(8) << num9 << endl;
22
23     return 0;
24 }
```

Mở rộng: Không gian tên

- ❑ Không gian tên
- ❑ Viết tắt toàn bộ và một phần không gian tên

Không gian tên và khai báo using

- Không gian tên giúp các thành phần lập trình khỏi bị trùng tên/đụng độ về tên (name conflict)
 - Tên đầy đủ của mỗi thành phần sẽ là:
`<tên không gian>::<tên thành phần>`
VD: `std::cout` là lệnh `cout` thuộc không gian `std`
- Có thể khai báo với C++ để viết tắt không gian tên:
 - `using namespace X`: viết tắt mọi thành phần thuộc `X`,
 - `Using namespace std`; viết tắt mọi thành phần thuộc không gian `std`, vd như `cin`, `cout`, `endl`,...
 - `using std::cout`: viết tắt một thành phần cụ thể, ở đây là viết tắt thành phần `cout` của `std`.

Ví dụ 7a: Viết tắt tên toàn không gian

```
1  // Fig. 1.4: fig01_04.cpp
2  // Printing a line with multiple statements.
3  #include <iostream>
4
5  using namespace std;
6  // function main begins program execution
7  int main()
8  {
9      cout << "Welcome ";
10     std::cout << "to C++!\n";
11
12     return 0;
13
14 } // end function main
```


Ví dụ 7b: Viết tắt tên từng thành phần

```
using std::cout;    // program uses cout
using std::cin;     // program uses cin
using std::endl;    // program uses endl
...
cout << "No need to write std::";
cin >> somevariable;
...
```

Tổng kết

- Tổng quan môi trường C++ và cách cài đặt
- Cấu trúc cơ bản chương trình C++
- Hàm cout, dùng kiểu lợp ngói, dùng với ký tự thoát
- Khai báo biến, chọn kiểu dữ liệu, cách đặt tên
- Nhập xuất, đọc ghi biến, tính toán trên biến
- Các chủ đề mở rộng: hằng, phong cách lập trình, không gian tên.

Trò chơi đoán số

- ❑ Mô tả chung
- ❑ Nhiệm vụ tuần này

Trò chơi đoán số

Máy tính nghĩ ra một số ngẫu nhiên trong khoảng từ 1 đến 1000, nó giữ bí mật và mời bạn đoán. Mỗi lần đoán, nó sẽ cho bạn thắng nếu bạn trả lời đúng số cần đoán. Nếu không đúng, bạn được biết giá trị vừa đoán là quá cao hay quá thấp so với số cần đoán. Sau 10 đoán mà vẫn không đoán trúng thì bạn sẽ thua.

Hello! What is your name?

Mai

Play with me, Mai! I am thinking of a number between 1 and 1000.

Take a guess.

10

Your guess is too high.

Take a guess.

2

Your guess is too low.

Take a guess.

4

Well done, Mai! You guessed my number in 3 guesses!

Trò chơi đoán số - Tuần 1

- Tuần này ta đã có thể làm được bước đầu tiên là in ra lời chào mừng, hỏi tên người chơi, giới thiệu trò chơi, và nhập dự đoán đầu tiên.
- Hãy viết một chương trình có tên là `GuessNumber` in ra lời chào sau:

```
Xin chao. Ten ban la gi?  
Mai  
Chao ban Mai. Toi dang nghi ve mot so trong khoang tu 1 den 1000.  
Ban hay thu doan!  
2  
Ban vua doan so 2.
```